

Program Name: Design and Implement a Lexical Analyzer using C Language

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

int isKeyword(char buffer[])
{
    char keyword[32][10] = {
        "auto", "break", "case", "char", "const",
        "continue", "default", "do", "double", "else",
        "enum", "extern", "float", "for", "goto",
        "if", "int", "long", "register", "return",
        "short", "signed", "sizeof", "static", "struct",
        "switch", "typedef", "union", "unsigned", "void",
        "volatile", "while"
    };

    int i;

    for (i = 0; i < 32; i++)
    {
        if (strcmp(keyword[i], buffer) == 0)
            return 1;
    }

    return 0;
}

int main()
{
    char ch;
    char buffer[50];

    char operators[] = "+-*/%=";
    char specialch[] = ";,(){}";

    FILE *fp;
    int i, j = 0;

    fp = fopen("input.dat", "r");
    if (fp == NULL)
    {
        fp = fopen("program.txt", "r");
    }

    if (fp == NULL)
    {
        printf("Error opening file\n");
        exit(0);
    }

    while ((ch = fgetc(fp)) != EOF)
    {
        /* Check operators */
```

```

for (i = 0; i < 6; i++)
{
    if (ch == operators[i])
    {
        printf("%c is special character\n", ch);
        break;
    }
}

/* Check special characters */
for (i = 0; i < 6; i++)
{
    if (ch == specialch[i])
    {
        printf("%c is special character\n", ch);
        break;
    }
}

/* Build a word or number */
if (isalnum(ch))
{
    buffer[j++] = ch;
}
else if (j != 0)
{
    buffer[j] = '\0';
    j = 0;

    /* Check whether token is a keyword */
    if (isKeyword(buffer))
    {
        printf("%s is keyword\n", buffer);
    }
    /* Check whether token is a number */
    else if (isdigit(buffer[0]))
    {
        int flag = 1;

        for (i = 0; buffer[i] != '\0'; i++)
        {
            if (!isdigit(buffer[i]))
            {
                flag = 0;
                break;
            }
        }

        if (flag)
            printf("%s is constant\n", buffer);
        else
            printf("%s is identifier\n", buffer);
    }
}
else
{
    printf("%s is identifier\n", buffer);
}
}

```

```

    }
}

/* Process last token if file doesn't end with whitespace */
if (j != 0)
{
    buffer[j] = '\0';

    if (isKeyword(buffer))
    {
        printf("%s is keyword\n", buffer);
    }
    else if (isdigit(buffer[0]))
    {
        printf("%s is constant\n", buffer);
    }
    else
    {
        printf("%s is identifier\n", buffer);
    }
}

fclose(fp);

return 0;
}

```

Output:**input.dat:**

```
int a=10;
float b=a+20;
if(b>30){
return b;
}
```

Output:

```
int is keyword
= is special character
a is identifier
; is special character
10 is constant
float is keyword
= is special character
b is identifier
+ is special character
a is identifier
; is special character
20 is constant
( is special character
if is keyword
b is identifier
) is special character
30 is constant
{ is special character
return is keyword
; is special character
b is identifier
} is special character
```

Program Name: Implement a Lexical Analyzer using LEX Tool

Program:

```
%{
#include <stdio.h>
%}

%%
int float for if else while {printf("keyword ");}
[a-zA-Z][a-zA-Z0-9]* {printf("identifier ");}
%%

int main() {
    printf("enter ip: ");
    yylex();
    return 0;
}

int yywrap() {
    return 1;
}
```

Output:

```
enter ip: int a float b
keyword identifier keyword identifier
```

Program Name: Display Number of Lines, Words, Spaces, and Characters using LEX

Program:

```
%{
#include <stdio.h>
int sc = 0, wc = 0, lc = 0, cc = 0;
%}

%%

[\n]    { lc++; cc += yyleng; }
[ \t]   { sc++; cc += yyleng; }
[^\t\n ]+ { wc++; cc += yyleng; }
%%

int main(int argc, char* argv[]) {
    printf("Enter the input:\n");
    yylex();
    printf("The number of lines=%d\n", lc);
    printf("The number of spaces=%d\n", sc);
    printf("The number of words=%d\n", wc);
    printf("The number of characters are=%d\n", cc);
    return 0;
}

int yywrap() {
    return 1;
}
```

Output:

```
Enter the input:
Hello world
This is compiler design lab
```

```
The number of lines=2
The number of spaces=5
The number of words=7
The number of characters are=40
```

Program Name: Convert Substring 'abc' to 'ABC' using LEX

Program:

```
%{  
#include <stdio.h>  
%}  
  
%%  
"abc" {printf("ABC");}  
. {putchar(yytext[0]);}  
%%  
  
int main() {  
    yylex();  
    return 0;  
}  
  
int yywrap() {  
    return 1;  
}
```

Output:

Input: welcome to abc lab

Output: welcome to ABC lab

Program Name: Count Vowels and Consonants using LEX

Program:

```
%{
#include <stdio.h>
int v = 0;
int c = 0;
}%

%%
[aeiouAEIOU] { v++; }
[a-zA-Z]      { c++; }
[ \t\n]       ;
.              ;
%%

int main() {
    printf("Enter string: ");
    yylex();
    printf("Vowels: %d\n", v);
    printf("Consonants: %d\n", c);
    return 0;
}

int yywrap() {
    return 1;
}
```

Output:

```
Enter string: Engineering
Vowels: 5
Consonants: 6
```

Program Name: Valid Arithmetic Expression using YACC

Program:

Lexer Specification (prg6.l):

```
%{
#include <stdio.h>
#include "y.tab.h"
extern int yylval;
}%

%%
[0-9]+ { yylval = atoi(yytext); return NUMBER; }
[ \t] ;
[\n] return 0;
. return yytext[0];
%%
```

```
int yywrap() { return 1; }
```

YACC Parser Specification (prg6.y):

```
%{
#include <stdio.h>
int flag = 0;
int yylex(void);
void yyerror();
}%

%token NUMBER
%left '+' '-'
%left '*' '/' '%'
%left '(' ')'

%%
ArithmeticExpression: E { printf("\nResult=%d\n", $$); return 0; };
E: E '+' E { $$ = $1 + $3; } | E '-' E { $$ = $1 - $3; }
  | E '*' E { $$ = $1 * $3; } | E '/' E { $$ = $1 / $3; }
  | E '%' E { $$ = $1 % $3; } | '(' E ')' { $$ = $2; }
  | NUMBER { $$ = $1; } ;

%%

void main() {
    printf("Enter an arithmetic expression: ");
    yyparse();
    if (flag == 0) printf("\nEntered Arithmetic expression is valid\n\n");
}

void yyerror() {
    printf("\nEntered Arithmetic expression is invalid\n\n");
    flag = 1;
}
```

Output:

Enter an arithmetic expression: $(5+3)*2-4/2$

Result=14

Entered Arithmetic expression is valid

Program Name: Valid Identifier using YACC

Program:

Lexer Specification (prg5.l):

```
%{
#include <stdio.h>
#include "y.tab.h"
%}

%%
[a-zA-Z_] return letter;
[0-9]    return digit;
.        return yytext[0];
\n      return 0;
%%

int yywrap() {
    return 1;
}
```

YACC Parser Specification (prg5.y):

```
%{
#include<stdio.h>
int valid=1;
int yylex(void);
int yyerror();
%}
%token digit letter
%%
start: letter s
s: letter s | digit s |;
%%

int yyerror() {
    printf("invalid identifier.\n");
    valid=0;
    return 0;
}

int main() {
    printf("enter identifier:");
    yyparse();
    if(valid)
        printf("Valid Identifier.\n");
    return 0;
}
```

Output:

enter identifier: abc
Valid Identifier.

Program Name: Calculator using LEX and YACC

Program:

Lexer Specification (calculator.l):

```
%{
/* Definition section */
#include<stdio.h>
#include<stdlib.h>
#include "y.tab.h"
extern int yylval;
}%

/* Rule Section */
%%
[0-9]+ {
    yylval=atoi(yytext);
    return NUMBER;
}
[ \t] ;
[\n] return 0;
. return yytext[0];
%%
```

```
int yywrap()
{
    return 1;
}
```

YACC Parser Specification (calculator.y):

```
%{
/* Definition section */
#include<stdio.h>
#include<stdlib.h>
int flag=0;
int yylex(void);
void yyerror();
}%
```

```
%token NUMBER
```

```
%left '+' '-'
%left '*' '/'
%left '(' ')'
/* Rule Section */
%%
```

```
ArithmeticExpression: E{
    printf("\nResult=%d\n", $$);
    return 0;
};
```

```
E:E+'E' {$$=$1+$3;}
E:E-'E' {$$=$1-$3;}
E:E'*E' {$$=$1*$3;}
E:E'/E' {$$=$1/$3;}
E:E'%E' {$$=$1%$3;}
E:'(E)' {$$=$2;}
```

```

| NUMBER {$$=$1;}
;
%%

//driver code
void main()
{
    printf("\nEnter Any Arithmetic Expression which can have operations Addition, Subtraction,
        Multiplication, Division, Modulus and Round brackets:\n");

    yyparse();
    if(flag==0)
        printf("\nEnter arithmetic expression is Valid\n\n");
}

void yyerror()
{
    printf("\nEnter arithmetic expression is Invalid\n\n");
    flag=1;
}

```

Output:

Enter Any Arithmetic Expression which can have operations Addition, Subtraction, Multiplication,
Division, Modulus and Round brackets:

15+4*3-(10/2)

Result=22

Entered arithmetic expression is Valid

Program Name: Epsilon Closure of States in NFA**Program:**

```
#include <stdio.h>
#include <string.h>

char states[20][20];
char result[20][20];
char copy[20];

int is_present(char state[20], int n)
{
    int i;
    for (i = 0; i < n; i++)
    {
        if (strcmp(result[i], state) == 0)
            return 1;
    }
    return 0;
}

void epsilon_closure(char start[20], FILE *INPUT)
{
    char current[20];
    char state1[20], input[20], state2[20];
    int count = 0;
    int i;

    strcpy(copy, start);
    strcpy(result[count++], start);
    i = 0;
    while (i < count)
    {
        strcpy(current, result[i]);
        rewind(INPUT);
        while (fscanf(INPUT, "%s %s %s", state1, input, state2) == 3)
        {
            if (strcmp(current, state1) == 0 && strcmp(input, "e") == 0)
            {
                if (!is_present(state2, count))
                {
                    strcpy(result[count++], state2);
                }
            }
        }
        i++;
    }

    printf("\nEpsilon closure of %s = {", copy);
    for (i = 0; i < count; i++)
    {
        printf("%s", result[i]);
        if (i < count - 1)
            printf(", ");
    }
    printf("}\n");
}
```

```

int main()
{
    FILE *INPUT;
    int n, i;

    INPUT = fopen("input.dat", "r");
    if (INPUT == NULL)
    {
        printf("Unable to open input.dat\n");
        return 1;
    }

    printf("Enter the no of states: ");
    scanf("%d", &n);
    printf("Enter the states: ");
    for (i = 0; i < n; i++)
    {
        scanf("%s", states[i]);
    }
    for (i = 0; i < n; i++)
    {
        epsilon_closure(states[i], INPUT);
    }
    fclose(INPUT);
    return 0;
}

```

Output:

input.dat:

```

0 e 1
1 e 2

```

Output:

```

Enter the no of states: 3
Enter the states: 0 1 2

```

Epsilon closure of 0 = {0, 1, 2}

Epsilon closure of 1 = {1, 2}

Epsilon closure of 2 = {2}

Program Name: Conversion of NFA to DFA**Program:**

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int st;
    struct node *link;
};

struct node1 {
    int nst[20];
};

void insert(int, char, int);
int findalpha(char);
void findfinalstate(void);
int insertdfastate(struct node1);
int compare(struct node1, struct node1);
void printnewstate(struct node1);

static int set[20], nostate, noalpha, s, notransition, nofinal, start, finalstate[20], r, buffer[20];
char c;
int complete = -1;
char alphabet[20];
static int eclosure[20][20] = {0};
struct node1 hash[20];
struct node *transition[20][20] = {NULL};

void main() {
    int i, j, k, m, t, n, l;
    struct node *temp;
    struct node1 newstate = {0}, tmpstate = {0};

    printf("\nEnter No of alphabets and alphabets:\n");
    scanf("%d", &noalpha);
    getchar();

    for (i = 0; i < noalpha; i++) {
        alphabet[i] = getchar();
        getchar();
    }

    printf("Enter the number of states:\n");
    scanf("%d", &nostate);

    printf("Enter the start state:\n");
    scanf("%d", &start);

    printf("Enter the number of final states:\n");
    scanf("%d", &nofinal);

    printf("Enter the final states:\n");
    for (i = 0; i < nofinal; i++) {
        scanf("%d", &finalstate[i]);
    }
}
```

```

printf("Enter no of transition:\n");
scanf("%d", &notransition);

printf("NOTE:- [Transition is in the form-> qno alphabet qno]\n");
printf("NOTE:- [States number must be greater than zero]\n");
printf("\nEnter transition:\n");

for (i = 0; i < notransition; i++) {
    scanf("%d %c%d", &r, &c, &s);
    insert(r, c, s);
}

for (i = 0; i < 20; i++) {
    for (j = 0; j < 20; j++) {
        hash[i].nst[j] = 0;
    }
}

complete = -1;
i = -1;

printf("\nEquivalent DFA.....\n");
printf("Transitions of DFA\n");

newstate.nst[start] = start;
insertdfastate(newstate);

while (i != complete) {
    i++;
    newstate = hash[i];

    for (k = 0; k < noalpha; k++) {
        c = 0;

        for (j = 1; j <= nostate; j++) {
            set[j] = 0;
        }

        for (j = 1; j <= nostate; j++) {
            l = newstate.nst[j];
            if (l != 0) {
                temp = transition[l][k];
                while (temp != NULL) {
                    if (set[temp->st] == 0) {
                        c++;
                        set[temp->st] = temp->st;
                    }
                    temp = temp->link;
                }
            }
        }
    }

    printf("\n");
    if (c != 0) {
        for (m = 1; m <= nostate; m++) {
            tmpstate.nst[m] = set[m];
        }
    }
}

```

```

        }
        insertdfastate(tmpstate);
        printnewstate(newstate);
        printf("%c\t", alphabet[k]);
        printnewstate(tmpstate);
        printf("\n");
    } else {
        printnewstate(newstate);
        printf("%c\t", alphabet[k]);
        printf("NULL\n");
    }
}
}

printf("\nStates of DFA:\n");
for (i = 0; i <= complete; i++) {
    printnewstate(hash[i]);
}

printf("\nAlphabets:\n");
for (i = 0; i < noalpha; i++) {
    printf("%c\t", alphabet[i]);
}

printf("\nStart State:\n");
printf("q%d", start);

printf("\nFinal states:\n");
findfinalstate();
}

int insertdfastate(struct node1 newstate) {
    int i;
    for (i = 0; i <= complete; i++) {
        if (compare(hash[i], newstate)) {
            return 0;
        }
    }
    complete++;
    hash[complete] = newstate;
    return 1;
}

int compare(struct node1 a, struct node1 b) {
    int i;
    for (i = 1; i <= nostate; i++) {
        if (a.nst[i] != b.nst[i]) {
            return 0;
        }
    }
    return 1;
}

void insert(int r, char c, int s) {
    int j;
    struct node *temp;
    j = findalpha(c);

```

```

    if (j == 999) {
        printf("error\n");
        exit(0);
    }
    temp = (struct node *)malloc(sizeof(struct node));
    temp->st = s;
    temp->link = transition[r][j];
    transition[r][j] = temp;
}

int findalpha(char c) {
    int i;
    for (i = 0; i < noalpha; i++) {
        if (alphabet[i] == c) {
            return i;
        }
    }
    return 999;
}

void findfinalstate() {
    int i, j, k;
    for (i = 0; i <= complete; i++) {
        for (j = 1; j <= nostate; j++) {
            for (k = 0; k < nofinal; k++) {
                if (hash[i].nst[j] == finalstate[k]) {
                    printnewstate(hash[i]);
                    printf("\t");
                    j = nostate;
                    break;
                }
            }
        }
    }
}

void printnewstate(struct node1 state) {
    int j;
    printf("{");
    for (j = 1; j <= nostate; j++) {
        if (state.nst[j] != 0) {
            printf("q%d,", state.nst[j]);
        }
    }
    printf("}\t");
}

```

Output:

Enter No of alphabets and alphabets:

2

a

b

Enter the number of states:

3

Enter the start state:

1

Enter the number of final states:

1

Enter the final states:

3

Enter no of transition:

4

NOTE:- [Transition is in the form-> qno alphabet qno]

NOTE:- [States number must be greater than zero]

Enter transition:

1 a 1

1 a 2

1 b 1

2 b 3

Equivalent DFA.....

Transitions of DFA

{q1,} a {q1,q2,}

{q1,} b {q1,}

{q1,q2,} a {q1,q2,}

{q1,q2,} b {q1,q3,}

{q1,q3,} a {q1,q2,}

{q1,q3,} b {q1,}

States of DFA:

{q1,} {q1,q2,} {q1,q3,}

Alphabets:

a b

Start State:

q1

Final states:

{q1,q3,}

Program Name: Minimization of DFA**Program:**

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_STATES 20
#define MAX_SYMBOLS 10

int transition[MAX_STATES][MAX_SYMBOLS];
int isFinal[MAX_STATES];

int stateCount, symbolCount;
int reachable[MAX_STATES];

void dfs(int state)
{
    if (reachable[state])
        return;

    reachable[state] = 1;

    for (int i = 0; i < symbolCount; i++)
    {
        int next = transition[state][i];

        if (next != -1)
            dfs(next);
    }
}

void removeUnreachable()
{
    for (int i = 0; i < stateCount; i++)
        reachable[i] = 0;

    dfs(0);
}

int equivalent(int s1, int s2, int group[])
{
    if (isFinal[s1] != isFinal[s2])
        return 0;

    for (int i = 0; i < symbolCount; i++)
    {
        int t1 = transition[s1][i];
        int t2 = transition[s2][i];

        if (t1 == -1 && t2 == -1)
            continue;

        if (t1 == -1 || t2 == -1)
```

```

        return 0;

        if (group[t1] != group[t2])
            return 0;
    }

    return 1;
}

void minimizeDFA()
{
    removeUnreachable();

    int group[MAX_STATES];
    int newGroup[MAX_STATES];

    for (int i = 0; i < stateCount; i++)
    {
        if (!reachable[i])
            group[i] = -1;
        else
            group[i] = isFinal[i];
    }

    int changed;

    do
    {
        changed = 0;

        for (int i = 0; i < stateCount; i++)
            newGroup[i] = -1;

        int nextGroup = 0;

        for (int i = 0; i < stateCount; i++)
        {
            if (group[i] == -1)
                continue;

            if (newGroup[i] != -1)
                continue;

            newGroup[i] = nextGroup;

            for (int j = i + 1; j < stateCount; j++)
            {
                if (group[j] != -1 &&
                    newGroup[j] == -1 &&
                    equivalent(i, j, group))
                {
                    newGroup[j] = nextGroup;
                }
            }
        }
    }

```

```

        nextGroup++;
    }

    for (int i = 0; i < stateCount; i++)
    {
        if (group[i] != newGroup[i])
            changed = 1;

        group[i] = newGroup[i];
    }
} while (changed);

int groupCount = 0;

for (int i = 0; i < stateCount; i++)
{
    if (group[i] >= groupCount)
        groupCount = group[i] + 1;
}

printf("\nMinimized DFA:\n");
printf("Total minimized states: %d\n", groupCount);

for (int i = 0; i < groupCount; i++)
{
    int final = 0;

    for (int j = 0; j < stateCount; j++)
    {
        if (group[j] == i && isFinal[j])
        {
            final = 1;
            break;
        }
    }

    printf("\nState %d", i);

    if (final)
        printf(" (Final)");

    printf(":\n");

    int representative = -1;

    for (int j = 0; j < stateCount; j++)
    {
        if (group[j] == i)
        {
            representative = j;
            break;
        }
    }
}

```

```

    if (representative != -1)
    {
        for (int s = 0; s < symbolCount; s++)
        {
            int target = transition[representative][s];

            if (target != -1)
            {
                printf(" On symbol %d -> State %d\n",
                    s, group[target]);
            }
        }
    }

    printf("\n");
}

int main()
{
    printf("Enter the number of states: ");
    scanf("%d", &stateCount);

    if (stateCount <= 0 || stateCount > MAX_STATES)
    {
        printf("Invalid number of states.\n");
        return 1;
    }

    printf("Enter number of symbols: ");
    scanf("%d", &symbolCount);

    if (symbolCount <= 0 || symbolCount > MAX_SYMBOLS)
    {
        printf("Invalid number of symbols.\n");
        return 1;
    }

    printf("Enter the transition (-1 for no transition):\n");

    for (int i = 0; i < stateCount; i++)
    {
        for (int j = 0; j < symbolCount; j++)
        {
            printf("Transition from state %d on symbol %d: ",
                i, j);

            scanf("%d", &transition[i][j]);

            if (transition[i][j] < -1 ||
                transition[i][j] >= stateCount)
            {
                printf("Invalid transition.\n");
                return 1;
            }
        }
    }
}

```

```

}

for (int i = 0; i < stateCount; i++)
    isFinal[i] = 0;

int numFinalStates;

printf("Enter the no. of final states: ");
scanf("%d", &numFinalStates);

if (numFinalStates < 0 || numFinalStates > stateCount)
{
    printf("Invalid number of final states.\n");
    return 1;
}

printf("Enter the final states:\n");

for (int i = 0; i < numFinalStates; i++)
{
    int f;
    scanf("%d", &f);

    if (f < 0 || f >= stateCount)
    {
        printf("Invalid final state.\n");
        return 1;
    }

    isFinal[f] = 1;
}

minimizeDFA();
return 0;
}

```

Output:

Enter the number of states: 5
Enter number of symbols: 2
Enter the transition (−1 for no transition):
Transition from state 0 on symbol 0: 1
Transition from state 0 on symbol 1: 2
Transition from state 1 on symbol 0: 1
Transition from state 1 on symbol 1: 3
Transition from state 2 on symbol 0: 1
Transition from state 2 on symbol 1: 2
Transition from state 3 on symbol 0: 1
Transition from state 3 on symbol 1: 4
Transition from state 4 on symbol 0: 1
Transition from state 4 on symbol 1: 2
Enter the no. of final states: 1
Enter the final states:
4

Minimized DFA:

Total minimized states: 4

State 0:

On symbol 0 → State 1
On symbol 1 → State 0

State 1:

On symbol 0 → State 1
On symbol 1 → State 2

State 2:

On symbol 0 → State 1
On symbol 1 → State 3

State 3 (Final):

On symbol 0 → State 1
On symbol 1 → State 0

Program Name: Computation of FIRST and FOLLOW Sets

Program:

```
#include <stdio.h>
#include <math.h>
#include <string.h>
#include <ctype.h>
#include <stdlib.h>

int n, m = 0, p, i = 0, j = 0;
char a[10][10], f[10];
void follow(char c);
void first(char c);

int main()
{
    int i, z;
    char c, ch;
    printf("Enter the no of productions:\n");
    scanf("%d", &n);
    printf("Enter the productions:\n");
    for (i = 0; i < n; i++)
        scanf("%s%c", a[i], &ch);
    do
    {
        m = 0;
        printf("Enter the elements whose first and follow is to be found:");
        scanf("%c", &c);
        first(c);
        printf("First(%c)={ ", c);
        for (i = 0; i < m; i++)
            printf("%c", f[i]);
        printf("}\n");
        strcpy(f, " ");
        m = 0;
        follow(c);
        printf("Follow(%c)={ ", c);
        for (i = 0; i < m; i++)
            printf("%c", f[i]);
        printf("}\n");
        printf("Continue(0/1)?");
        scanf("%d%c", &z, &ch);
    } while (z == 1);
    return (0);
}

void first(char c)
{
    int k;
    if (!isupper(c))
        f[m++] = c;
    for (k = 0; k < n; k++)
    {
        if (a[k][0] == c)
        {
            if (a[k][2] == '#')
                f[m++] = '#';
        }
    }
}
```

```

        else if (islower(a[k][2]))
            f[m++] = a[k][2];
        else
            first(a[k][2]);
    }
}

void follow(char c)
{
    if (a[0][0] == c)
        f[m++] = '$';
    for (i = 0; i < n; i++)
    {
        for (j = 2; j < strlen(a[i]); j++)
        {
            if (a[i][j] == c)
            {
                if (a[i][j + 1] != '\0')
                    first(a[i][j + 1]);
                if (a[i][j + 1] == '\0' && c != a[i][0])
                    follow(a[i][0]);
            }
        }
    }
}

```

Output:

Enter the no of productions:

3

Enter the productions:

S=Ab

A=a

A=#

Enter the elements whose first and follow is to be found: S

First(S)={a#}

Follow(S)={\$}

Continue(0/1)? 1

Enter the elements whose first and follow is to be found: A

First(A)={a#}

Follow(A)={b}

Continue(0/1)? 0

Program Name: Recursive Descent Parser

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char input[20];
int i = 0, error = 0;
void E(), T(), Eprime(), Tprime(), F();

int main() {
    printf("Enter an arithmetic expression: ");
    gets(input);
    E();
    if (strlen(input) == i && error == 0)
        printf("\nAccepted\n");
    else
        printf("\nRejected\n");
    return 0;
}

void E() { T(); Eprime(); }

void Eprime() {
    if (input[i] == '+') { i++; T(); Eprime(); }
}

void T() { F(); Tprime(); }

void Tprime() {
    if (input[i] == '*') { i++; Tprime(); }
}

void F() {
    if (isdigit(input[i]))
        i++;
    else if (input[i] == '(') {
        i++; E();
        if (input[i] == ')') i++;
        else error = 1;
    } else error = 1;
}
```

Output:

Enter an arithmetic expression: a+b
Accepted

Program Name: Shift Reduce Parser

Program:

```
#include <stdio.h>
#include <string.h>

struct ProductionRule
{
    char left[10];
    char right[10];
};

int main()
{
    char input[20], stack[50], temp[50], ch[2], *token1, *token2, *substring;
    int i, j, stack_length, substring_length, stack_top, rule_count = 0;
    struct ProductionRule rules[10];

    stack[0] = '\0';

    printf("\nEnter the number of production rules:");
    scanf("%d", &rule_count);

    printf("\nEnter the number of production rules(in the form of left->right):\n");
    for (i = 0; i < rule_count; i++)
    {
        scanf("%s", temp);
        token1 = strtok(temp, "->");
        token2 = strtok(NULL, "->");
        strcpy(rules[i].left, token1);
        strcpy(rules[i].right, token2);
    }

    printf("\nEnter the input string:");
    scanf("%s", input);

    i = 0;
    while (1)
    {
        if (i < strlen(input))
        {
            ch[0] = input[i];
            ch[1] = '\0';
            i++;
            strcat(stack, ch);
            printf("%s\t", stack);
            for (int k = i; k < strlen(input); k++)
            {
                printf("%c", input[k]);
            }
            printf("\tShift %s\n", ch);
        }

        for (j = 0; j < rule_count; j++)
        {
            substring = strstr(stack, rules[j].right);
            if (substring != NULL)
```

```

    {
        stack_length = strlen(stack);
        substring_length = strlen(substring);
        stack_top = stack_length - substring_length;
        stack[stack_top] = '\0';
        strcat(stack, rules[j].left);
        printf("%s\t", stack);
        for (int k = i; k < strlen(input); k++)
        {
            printf("%c", input[k]);
        }
        printf("\tReduce %s-> %s\n", rules[j].left, rules[j].right);
        j = -1;
    }
}

if (strcmp(stack, rules[0].left) == 0 && i == strlen(input))
{
    printf("\nAccepted\n");
    break;
}

if (i == strlen(input))
{
    printf("\nNot Accepted\n");
    break;
}
}

return 0;
}

```

Output:

Enter the number of production rules: 3

Enter the number of production rules(in the form of left->right):

E->E+E

E->E*E

E->a

Enter the input string: a+a*a

a	+a*a	Shift a
E	+a*a	Reduce E-> a
E+	a*a	Shift +
E+a	*a	Shift a
E+E	*a	Reduce E-> a
E	*a	Reduce E-> E+E
E*	a	Shift *
E*a		Shift a
E*E		Reduce E-> a
E		Reduce E-> E*E

Accepted

Program Name: Constant Propagation (Code Optimization)

Program:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

void input();
void output();
void change(int p,char *res);
void constant();

struct expr
{
    char op[2],op1[5],op2[5],res[5];
    int flag;
}arr[10];

int n;

void main()
{
    input();
    constant();
    output();
}

void input()
{
    int i;
    printf("\n\nEnter the number of expressions: ");
    scanf("%d",&n);
    printf("Enter the input: \n");
    for(i=0;i<n;i++)
    {
        scanf("%s%s%s%s",arr[i].op,arr[i].op1,arr[i].op2,arr[i].res);
        arr[i].flag=0;
    }
}

void constant()
{
    int i;
    int op1,op2,res;
    char op,res1[5];
    for(i=0;i<n;i++)
    {
        if(isdigit(arr[i].op1[0]) && isdigit(arr[i].op2[0]) || strcmp(arr[i].op,"")==0)
        {
            op1=atoi(arr[i].op1);
            op2=atoi(arr[i].op2);
            op=arr[i].op[0];
            switch(op)
            {
                case '+': res=op1+op2; break;
                case '-': res=op1-op2; break;
```

```

        case '*': res=op1*op2; break;
        case '/': res=op1/op2; break;
        case '=': res=op1; break;
    }
    sprintf(res1,"%d",res);
    arr[i].flag=1;
    change(i,res1);
}
}

void output()
{
    int i=0;
    printf("\nThe optimized code is:\n");
    for(i=0;i<n;i++)
    {
        if(!arr[i].flag)
        {
            printf("%s %s %s %s\n",arr[i].op,arr[i].op1,arr[i].op2,arr[i].res);
        }
    }
}

void change(int p,char *res)
{
    int i;
    for(i=p+1;i<n;i++)
    {
        if(strcmp(arr[p].res,arr[i].op1)==0)
        {
            strcpy(arr[i].op1,res);
        }
        else if(strcmp(arr[p].res,arr[i].op2)==0)
        {
            strcpy(arr[i].op2,res);
        }
    }
}

```

Output:

Enter the number of expressions: 3

Enter the input:

= 3 - a

+ a t1 t2

* t2 a t3

The optimized code is:

+ 3 t1 t2

* t2 3 t3

Program Name: Intermediate Code Generation (Three-Address Code)

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int isp(char item);
void output(char item);
void push(char item);
char pop(void);
void quad(void);

char exp[20];
char res[20];
char a[20];
int top = 0, z = 0, i = 0, l;
char x, item;

void main()
{
    printf("Enter the infix expression: ");
    gets(exp);
    l = strlen(exp);
    push('#');

    while ((item = exp[i]) != '\0')
    {
        if (isalpha(exp[i]))
            output(item);
        else if (item == '+' || item == '-' || item == '*' || item == '/' || item == '^')
            push(item);
        else if (item == '(')
            push(item);
        else if (item == ')')
        {
            while ((x = pop()) != '(')
                output(x);
        }
        else if (isp(x = pop()) < isp(item))
        {
            push(x);
            push(item);
        }
        else
        {
            output(x);
            push(item);
        }
        i++;
    }

    while ((x = pop()) != '#')
        output(x);

    printf("Postfix expression: ");
    puts(res);
}
```

```

    quad();
}

int isp(char item)
{
    if ((item == '+' ) || (item == '-'))
        return 1;
    else if ((item == '*' ) || (item == '/'))
        return 2;
    else if ((item == '^'))
        return 3;
    else
        return 0;
}

void output(char item)
{
    res[z++] = item;
}

void push(char item)
{
    a[++top] = item;
}

char pop(void)
{
    item = a[top--];
    return item;
}

void quad()
{
    int i, x = 0;
    char m, n, temp, str1[5], str2[5];
    printf("\nOperator\ttop1\ttop2\tresult\n");
    printf("-----");

    for (i = 0; i < z; i++)
    {
        if (isalnum(res[i]))
        {
            push(res[i]);
        }
        else
        {
            if (isalpha(m = pop()))
            {
                str1[0] = m;
                str1[1] = '\0';
            }
            else
            {
                str1[0] = 't';
                str1[1] = m;
                str1[2] = '\0';
            }
        }
    }
}

```

```

        if (isalpha(n = pop()))
        {
            str2[0] = n;
            str2[1] = '\0';
        }
        else
        {
            str2[0] = 't';
            str2[1] = n;
            str2[2] = '\0';
        }

        x++;
        printf("\n%-8c\t%-7s\t%-7s\tt%d\n", res[i], str2, str1, x);
        temp = x + '0';
        push(temp);
    }
}

```

Output:

Enter the infix expression: a+(b*c)-d

Postfix expression: abc*d-+

Operator	op1	op2	result
*	b	c	t1
-	t1	d	t2
+	a	t2	t3